kubectl-ai: Natural Language Operations for Kubernetes (AI Powered K8s Assistant)

Bo-Yi Wu (GDE)
https://blog.wu-boy.com/

2025/10/23 16:00 - 16:40

KubeSummit 2025

kubectl-ai Repo

Outline

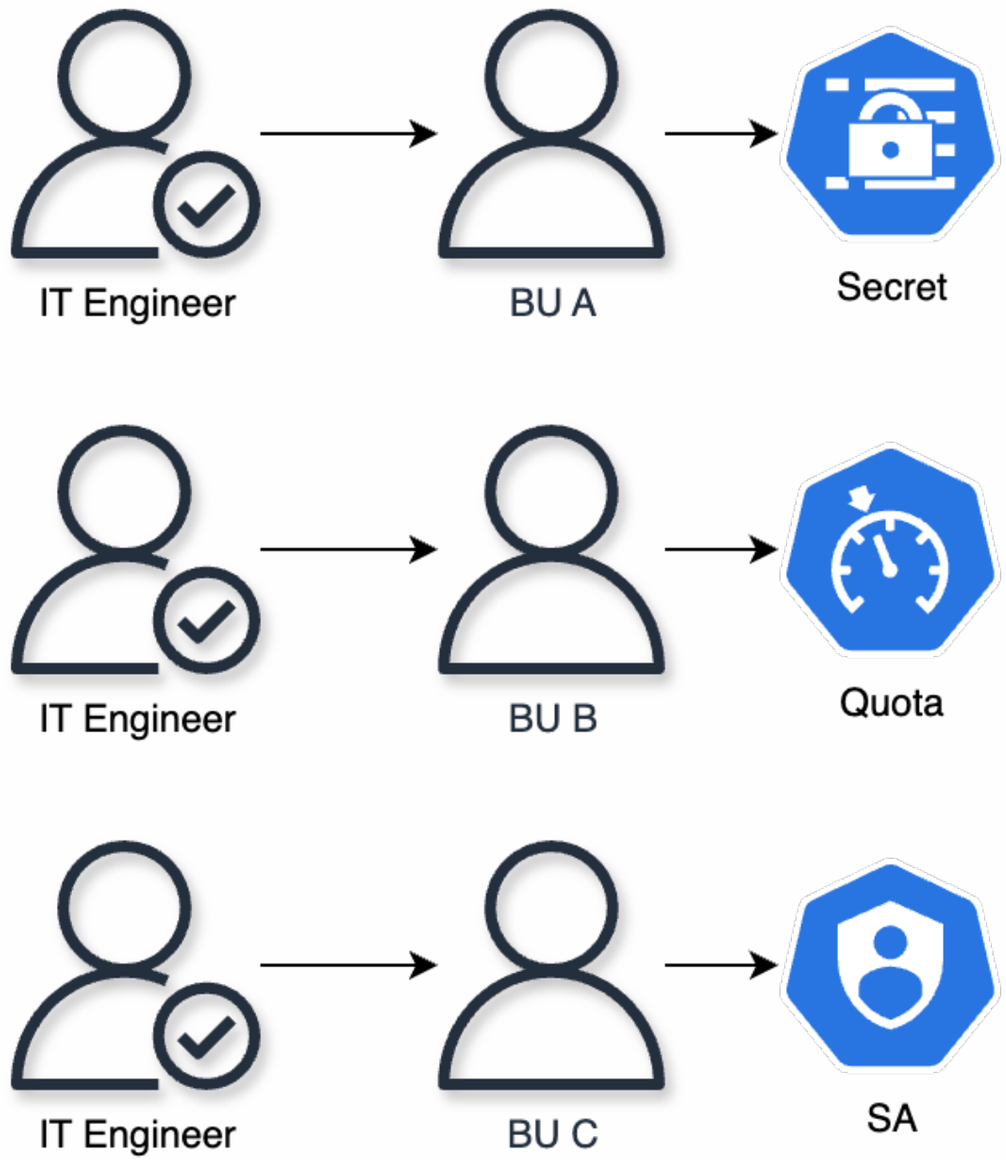
- Why do we use the kuebectl-ai Assistant?
- Demo of three core features
- Technical Architecture

Who has ever spent over 30 minutes troubleshooting why a Pod failed to start? (kubectl cheat sheet?)

https://kubernetes.io/docs/reference/kubectl/quick-reference/

Multiple teams are simultaneously reaching out to IT engineers for help resolving Kubernetes issues.

Terminal 1: kubectl get pods -n prod
Terminal 2: kubectl describe pod nginx-xxx
Terminal 3: kubectl logs nginx-xxx --previous
Terminal 4: kubectl get events -n prod
Terminal 5: kubectl get service nginx
Terminal 6: curl -v http://nginx-service



kubectl flow vs. kubectl-ai

Traditional Method	kubectl-ai
🔍 Need to remember 20+ kubectl commands	💬 Natural language conversation
🔁 Manually execute multiple related commands	🤖 AI automatic reasoning and decision-making
📝 Need to write complex YAML	✨ Just describe your needs
⏱️ Takes 15–30 minutes to troubleshoot	⚡ Get answers in 2–5 minutes

What is kubectl-ai?

What is kubectl-ai?

- ✅ AI plugin for kubectl
- ✅ Supports 8+ mainstream LLMs (Gemini/GPT-4/Claude/Grok)
- ✅ Open source (Apache 2.0) - Google Cloud Platform
- ✅ Over 95% evaluation success rate
- ✅ MCP protocol for integration with third-party tools

Latest Benchmark Results (August 2025)

Comprehensive evaluation on identical 10-task Kubernetes benchmark with proper CNI environment:

Model	Success	Fail	Success Rate
gemini-2.5-flash-preview-04-17	10	0	100%
gemini-2.5-pro-preview-03-25	10	0	100%
AWS Bedrock Claude 3.7 Sonnet	10	0	100%
AWS Bedrock Claude Sonnet 4	10	0	100%
gemma-3-27b-it	8	2	80%

https://github.com/GoogleCloudPlatform/kubectl-ai/blob/main/k8s-bench.md

Demo of Three Core Features

- Intelligent Troubleshooting
- MCP Client Integration
- MCP Server Capability

Demo 01
(Intelligent Troubleshooting)

Scenario: Newly deployed nginx service is not functioning properly
Objective: Demonstrate how AI can quickly identify the issue and suggest solutions

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:wrong-tag  # wrong tag
        resources:
          requests:
            memory: "10Gi"      # wrong value
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
  - port: 80
```

```
kubectl-ai \
"What is wrong with the Nginx service? Why is it not accessible?"
```

```
$ kubectl-ai --skip-permissions "What is wrong with the nginx service? Why is it not accessible?"

 I'll help you diagnose why the nginx service is not accessible. Let me start
 by examining the current state of the nginx service and related resources.


 Running: kubectl get services --all-namespaces | grep -i nginx

 Running: kubectl get pods --all-namespaces | grep -i nginx

 Running: kubectl get deployments --all-namespaces | grep -i nginx

 I see there are permission restrictions. Let me check what namespaces you
 have access to and then look for nginx resources in those specific
 namespaces.


 Running: kubectl get namespaces

 Running: kubectl get services

 Running: kubectl get pods

 Running: kubectl get deployments
```

Problem Identified

Found the Issue! 🔍

The nginx service is not accessible because no pods are running. The pods are failing to be created due to a Resource Quota Exceeded error.

Root Cause:

Your namespace has a `compute-quota` ResourceQuota that limits total memory to 6Gi, and currently 5Gi is already used. The nginx deployment is trying to create 3 replicas, each requesting 10Gi of memory, which far exceeds the available quota.

Error Message:

```
  exceeded quota: compute-quota, requested: limits.memory=10Gi, used: limits.
memory=5Gi, limited: limits.memory=6Gi
```

Current State:

- Service: nginx-svc exists but has no endpoints (no pods to route traffic to)
- Deployment: Wants 3 replicas, but 0 are available
- ReplicaSet: Cannot create any pods due to quota
- Namespace Quota: 5Gi used out of 6Gi limit (only 1Gi available)

Solutions from kubectl-ai

```
### Solutions:

Option 1: Reduce Memory Request (Recommended)

  kubectl patch deployment nginx --patch
'{"spec":{"template":{"spec":{"containers":[{"name":"nginx",
"resources":{"limits":{"memory":"256Mi"},
"requests":{"memory":"128Mi"}}}]}}}}'

Option 2: Reduce Replica Count

  kubectl scale deployment nginx --replicas=1
  # Then reduce memory per pod to fit quota

Option 3: Request Quota Increase
Contact your cluster administrator to increase the memory quota for your
namespace.

Would you like me to fix this by reducing the memory limits to a more
reasonable value (like 256Mi per pod)? 🔧
```

Problem Resolved from kubectl-ai

✅ Problem Resolved!

What was wrong:

1. Resource Quota Issue: The deployment requested 10Gi memory per pod, but the namespace only had 6Gi total limit with 5Gi already used
2. Image Not Found: The custom image `mtkomcr.mediatek.inc/public/nginx:stable1` didn't exist in the registry

What was fixed:

1. ✅ Reduced memory limits from 10Gi to 256Mi per pod (within quota)
2. ✅ Changed image to `nginx:stable` from Docker Hub
3. ✅ Added CPU requests (100m) for better resource management

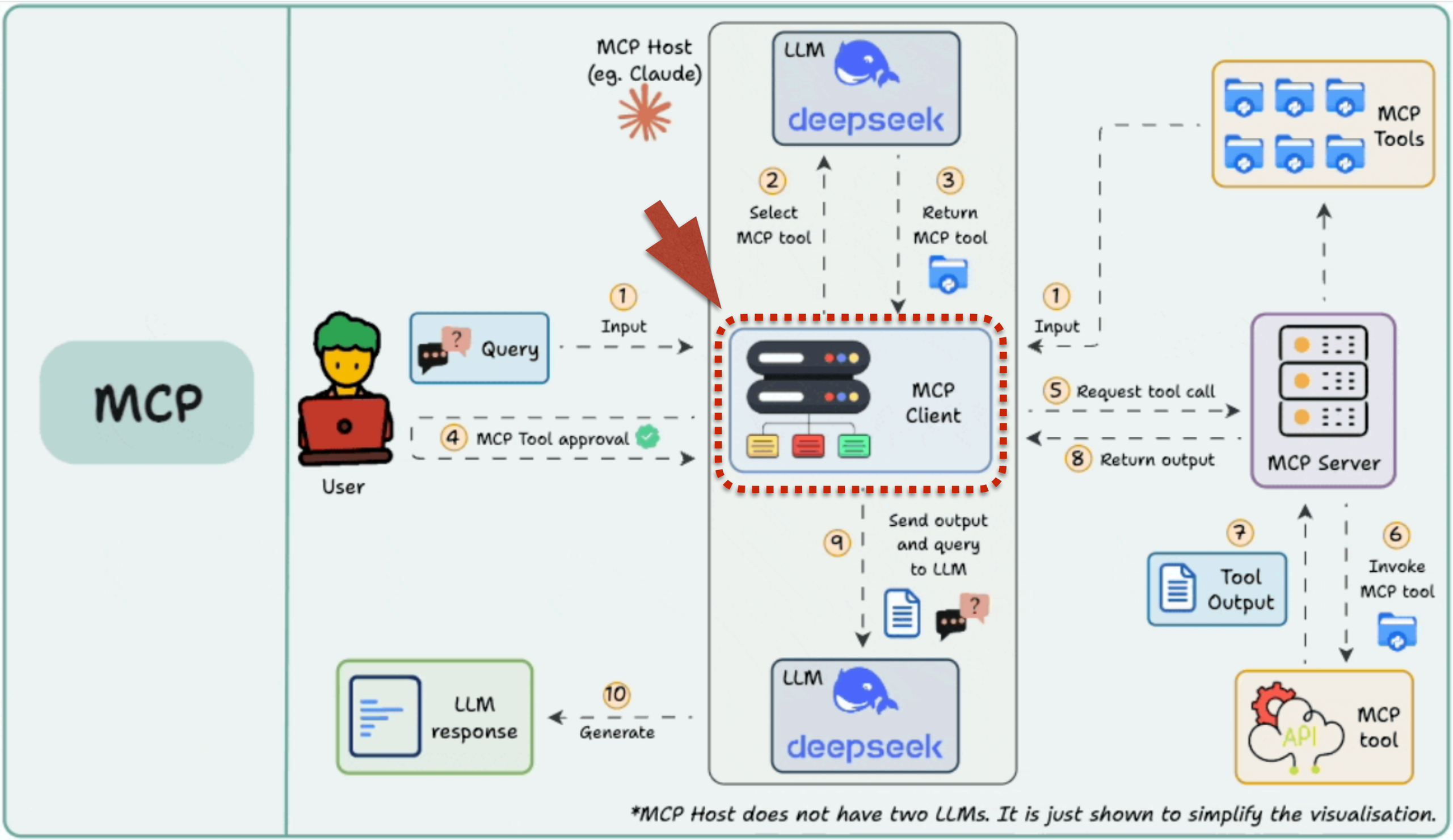
Current Status:

- Pods: 3/3 Running ✅
- Deployment: 3/3 Ready ✅
- Service: nginx-svc has 3 healthy endpoints ✅
- Endpoints: `192.168.106.23:80, 192.168.77.217:80, 192.168.89.229:80`

Your nginx service is now accessible at `192.168.57.73:80` (ClusterIP) from within the cluster! 🚀

Demo 02
(MCP Client Integration)

What is MCP Client?



Challenge & Solution for Kubernetes Security Audits

Challenge:

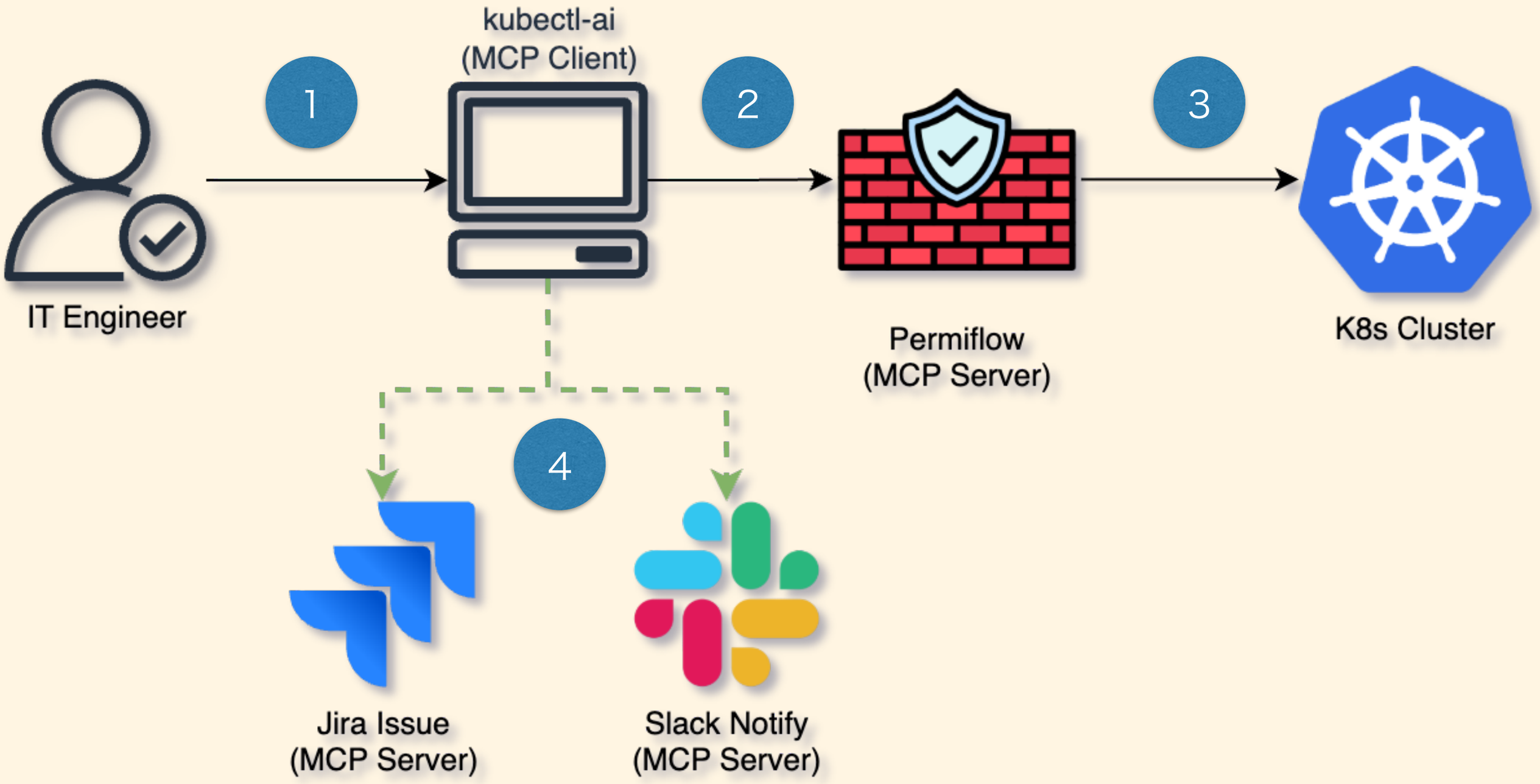
- Manual execution of multiple security tools across systems
- Time-consuming data collection and correlation by hand
- Manual report creation and distribution
- High risk of human error and resource consumption

Solution: One-command orchestration across multiple MCP servers

- Single command triggers automated security tools seamlessly
- Automatic data aggregation and unified result reporting
- Effortless report generation and distribution
- Minimized manual intervention for higher efficiency and accuracy
- Scalable to multi-server environments

Benefits:

- Greatly increased audit speed and reliability
- Lower human error and operational costs
- Enhanced automation and standardization of security monitoring


kubectl-ai
(MCP Client)
1
2
3
IT Engineer
Permiflow
(MCP Server)
K8s Cluster
4
Jira Issue
(MCP Server)
Slack Notify
(MCP Server)

kubectl-ai –mcp-client \
"Scan the RBAC permissions in the srv-gitea namespace, identify overprivileged ServiceAccounts, and create a Jira issue in the GAIA project with a summary of the findings included in the description."

The following commands require your approval to run:

- kubectl get serviceaccounts -n srv-gitea
- [MCP: permiflow] permiflow_scan_rbac(format=json)

Do you want to proceed ?

Use MCP Tool: permiflow

1. Yes
2. Yes, and don't ask me again
3. No

Enter your choice: 2

Running: kubectl get serviceaccounts -n srv-gitea

Running: [MCP: permiflow] permiflow_scan_rbac(format=json)

Here are the findings of the RBAC scan for the srv-gitea namespace:

Namespace ServiceAccounts:

- default
- gitea-runner

RBAC scan summary for the cluster:

- ClusterAdminBindings: 0 (no accounts have cluster-admin privileges)
- WildcardVerbs: 0 (no resources with wildcard/“*” verb usage)
- SecretsAccess: 0 (no direct secret access detected)
- PrivilegeEscalation: 0 (no privilege escalation risk detected)
- ExecAccess: 0 (no potentially dangerous exec access detected)
- ConfigReadSecrets: 0 (no ConfigMap/Secret read overprivilege detected)

No overprivileged service accounts were detected in the srv-gitea namespace or cluster.

https://github.com/tutran-se/permiflow

Proceeding to create a Jira issue in the GAIA project with this summary.

Running: [MCP: jira-server] jira-server_jira_create_issue(description=RBAC scan conducted for the srv-gitea namespace.

Namespace ServiceAccounts:
- default
- gitea-runner

RBAC scan summary:
- ClusterAdminBindings: 0 (no accounts have cluster-admin privileges)
- WildcardVerbs: 0 (no resources with wildcard/“*” verb usage)
- SecretsAccess: 0 (no direct secret access detected)
- PrivilegeEscalation: 0 (no privilege escalation risk detected)
- ExecAccess: 0 (no potentially dangerous exec access detected)
- ConfigReadSecrets: 0 (no ConfigMap/Secret read overprivilege detected)

No overprivileged service accounts were detected in the srv-gitea namespace or cluster., issue_type=Task, project_key=GAIA, summary=RBAC Scan in srv-gitea Namespace: No Overprivileged ServiceAccounts Detected)

Use MCP Tool: create Jira issue

RBAC scan completed for the srv-gitea namespace:

ServiceAccounts found:

- default
- gitea-runner

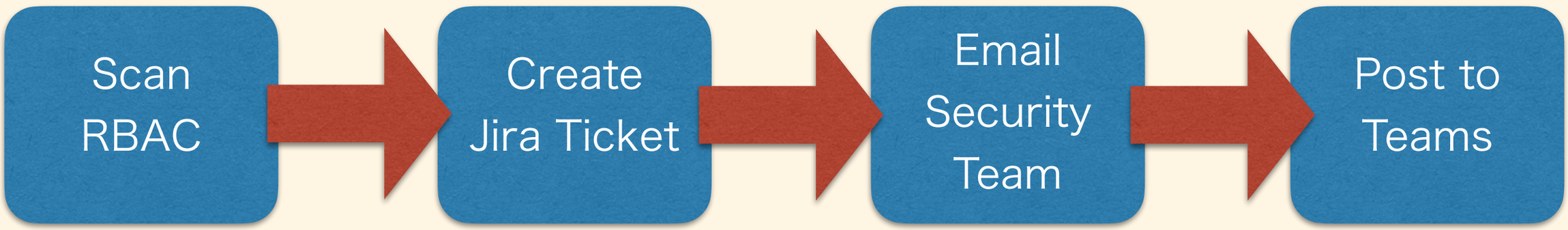
Summary of RBAC privileges:

- No service account has cluster-admin privileges
- No wildcard verb usage
- No direct access to secrets
- No detected privilege escalation vectors
- No exec access risks
- No overprivileged ConfigMap/Secret reading

A Jira issue has been created in the GAIA project with these findings:

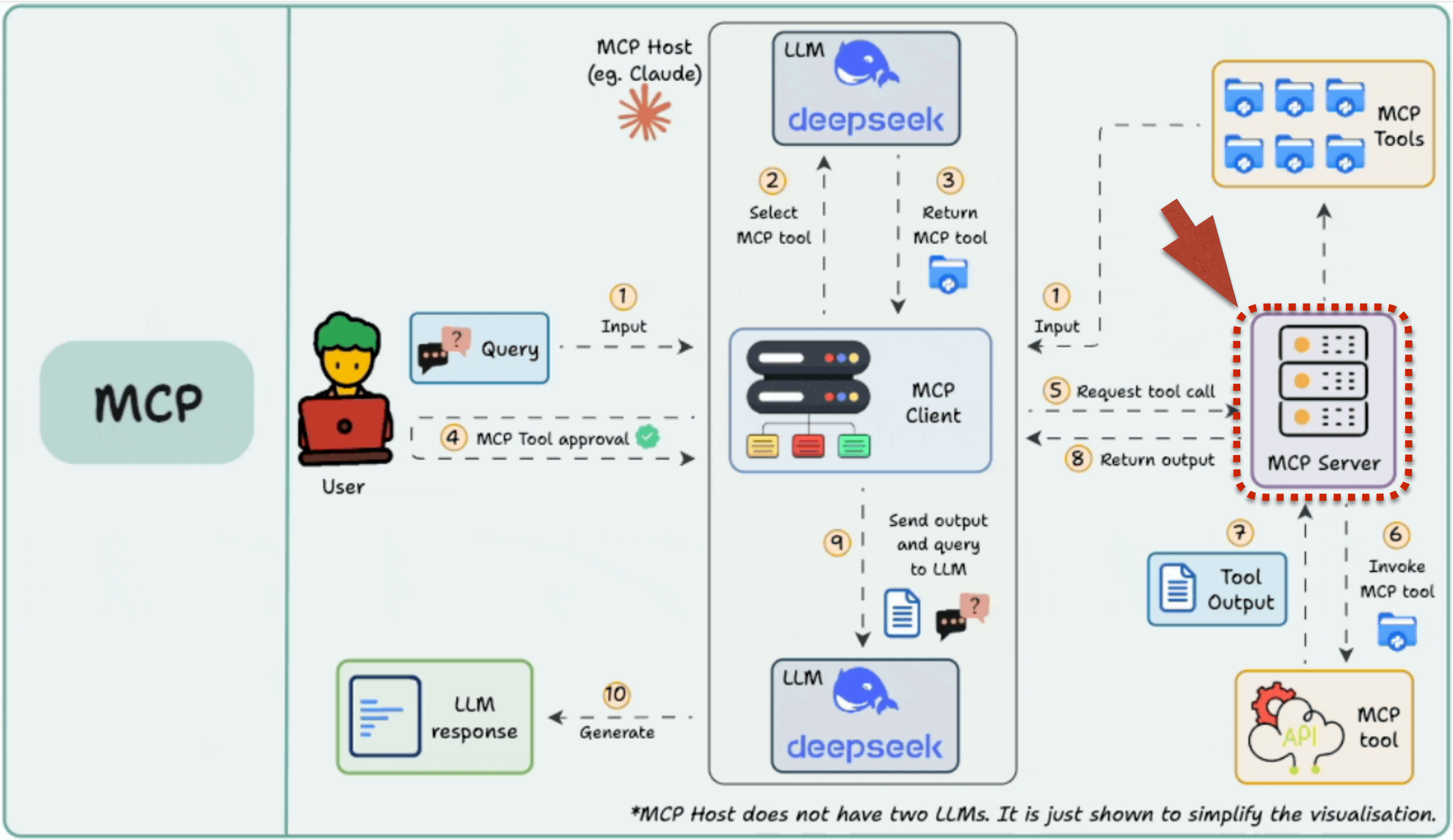
- Issue Key: GAIA-1459
- Status: Backlog
- Summary: RBAC Scan in srv-gitea Namespace: No Overprivileged ServiceAccounts Detected

Advanced Workflows



Demo 03
(MCP Server Capability)

What is MCP Server?



```
kubectl-ai –mcp-server \
–mcp-server-mode \
streamable-http \
–http-port 9080
```

Claude Code CLI integrate with 1. Permiflow MCP 2. kubectl-ai MCP

Prompt

"Scan the RBAC permissions in the srv-gitea namespace, identify overprivileged ServiceAccounts"

Overprivileged ServiceAccount Identified:

🚨 **HIGH RISK: `gitea-runner` ServiceAccount**

Permissions via Role `mke-deploy`:
- `configmaps: [*]` - Full access to ConfigMaps
- `pods: [*]` - Full access to Pods
- `secrets: [*]` - **CRITICAL: Full access to all Secrets**
- `services: [*]` - Full access to Services
- `deployments.apps: [*]` - Full access to Deployments
- `statefulsets.apps: [*]` - Full access to StatefulSets
- `horizontalpodautoscalers.autoscaling: [*]` - Full HPA access
- `httpproxies.projectcontour.io: [*]` - Full Contour proxy access

Security Risks:

1. **Secrets Access**: The `gitea-runner` can read/write ALL secrets in the namespace, including:
   - Database credentials
   - API keys
   - TLS certificates
   - Authentication tokens
2. **Pod Control**: Can create/delete/modify pods, potentially:
   - Escalating privileges through pod specs
   - Mounting sensitive volumes
   - Running privileged containers
3. **Deployment Control**: Full control over workload deployments could lead to:
   - Injecting malicious containers
   - Modifying security contexts
   - Changing resource limits

Recommendations:

1. **Apply Principle of Least Privilege**: Replace wildcard (`*`) verbs with specific actions
2. **Restrict Secret Access**: Limit to specific secrets needed for CI/CD
3. **Scope Resource Access**: Only grant access to resources the runner actually needs
4. **Consider Pod Security Standards**: Implement pod security policies to prevent privilege escalation

The `gitea-runner` ServiceAccount has excessive permissions that could be exploited if compromised.

```
kubectl-ai –mcp-server \
  –mcp-server-mode \
    streamable-http \
      –http-port 9080 \
        –external-tools
```

- All kubectl-ai built-in tools (kubectl, cluster analysis, etc.)
- Tools from external MCP servers (filesystem, web search, etc.)
- Unified interface for all tools through a single MCP endpoint

Demo
(kubectl-ai MCP server Acts as a Tool Aggregator)

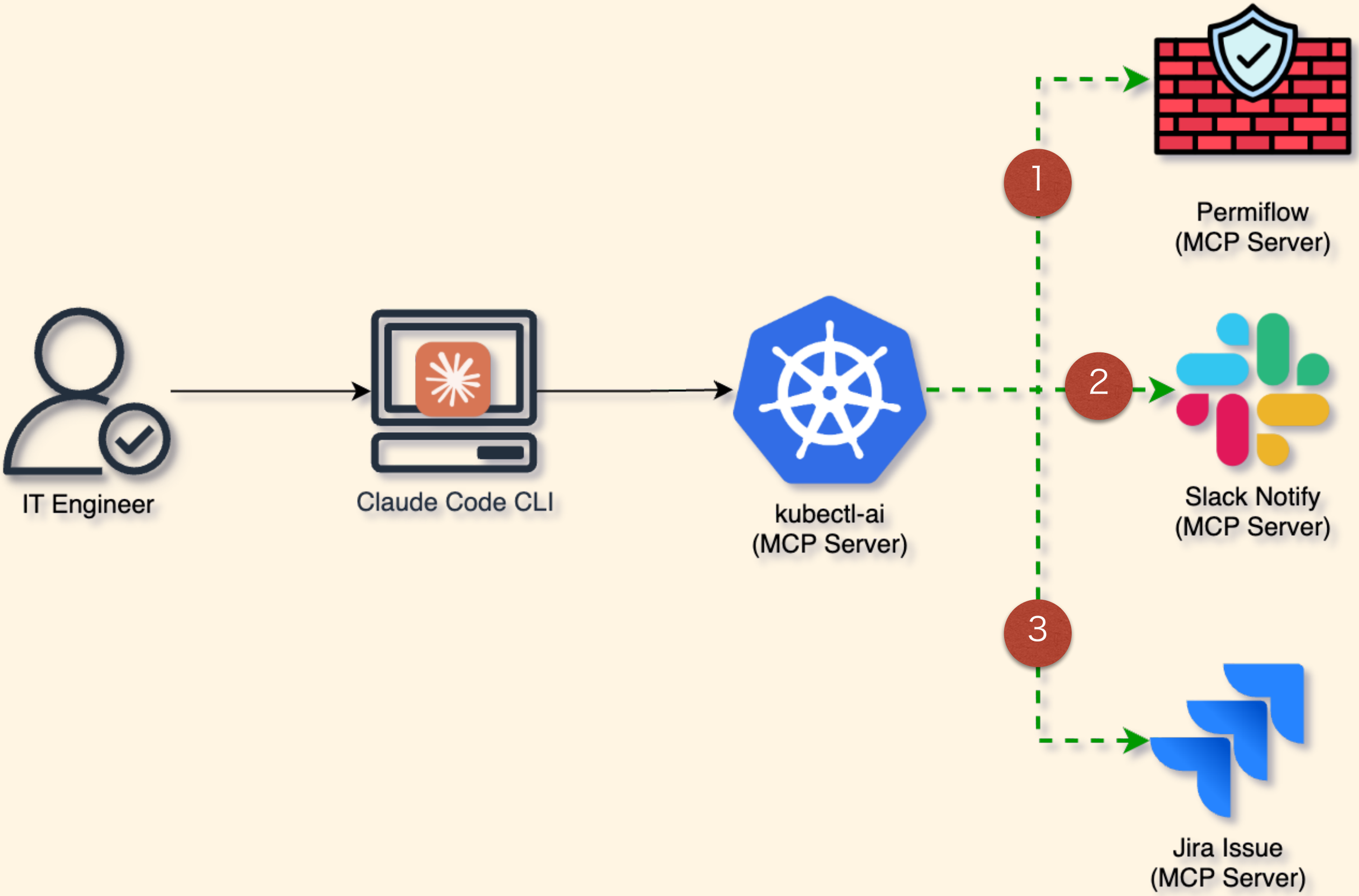
Kubernetes MCP Server

Status: ✔ connected
URL: http://localhost:9080/mcp
Config location: /Users/mtk10671/.claude.json
Capabilities: tools
Tools: 3 tools

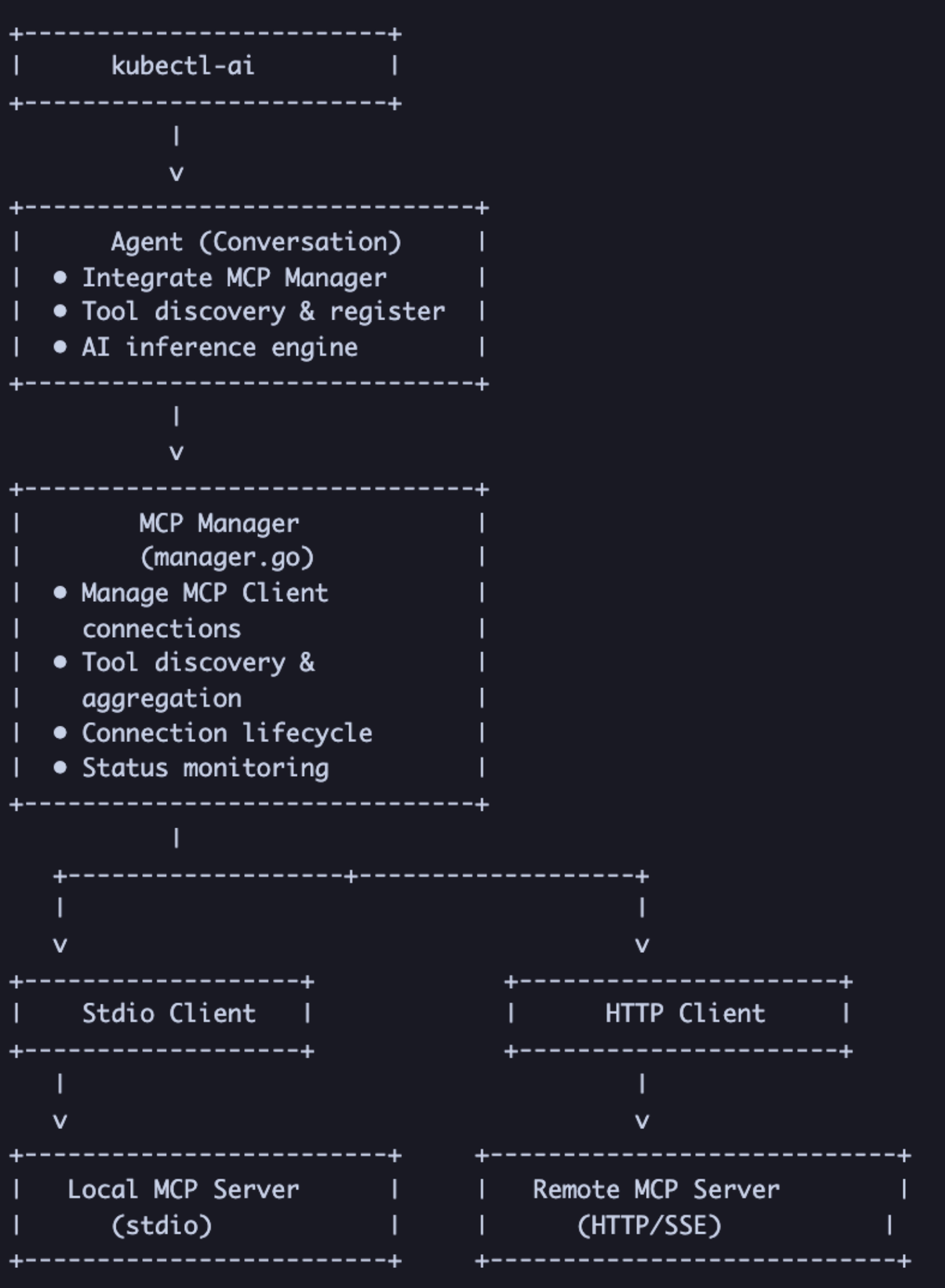
❯ 1. View tools
  2. Authenticate
  3. Reconnect

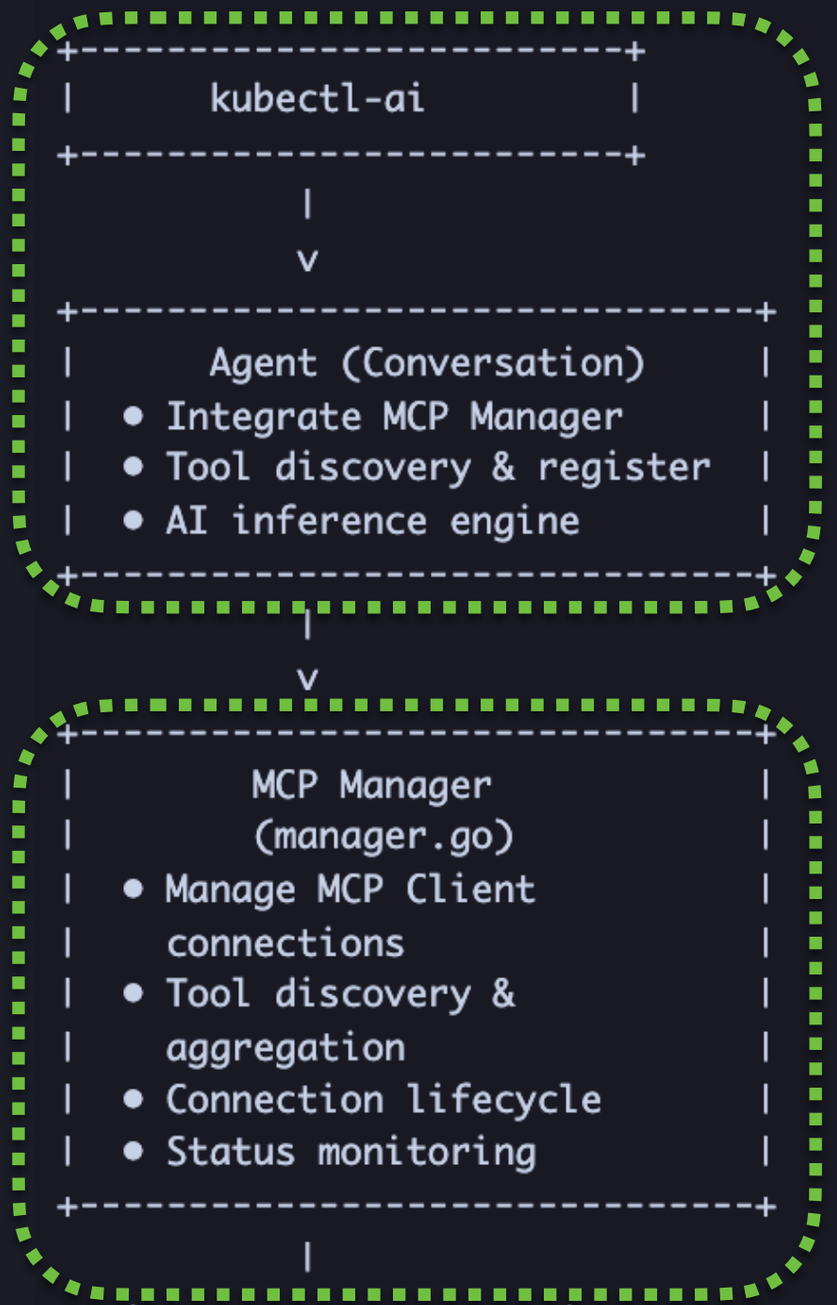
Tools for kubernetes (3 tools)

  1. bash
  2. kubectl
❯ 3. permiflow_scan_rbac


Permiflow
(MCP Server)
1
IT Engineer
Claude Code CLI
kubectl-ai
(MCP Server)
2
Slack Notify
(MCP Server)
3
Jira Issue
(MCP Server)

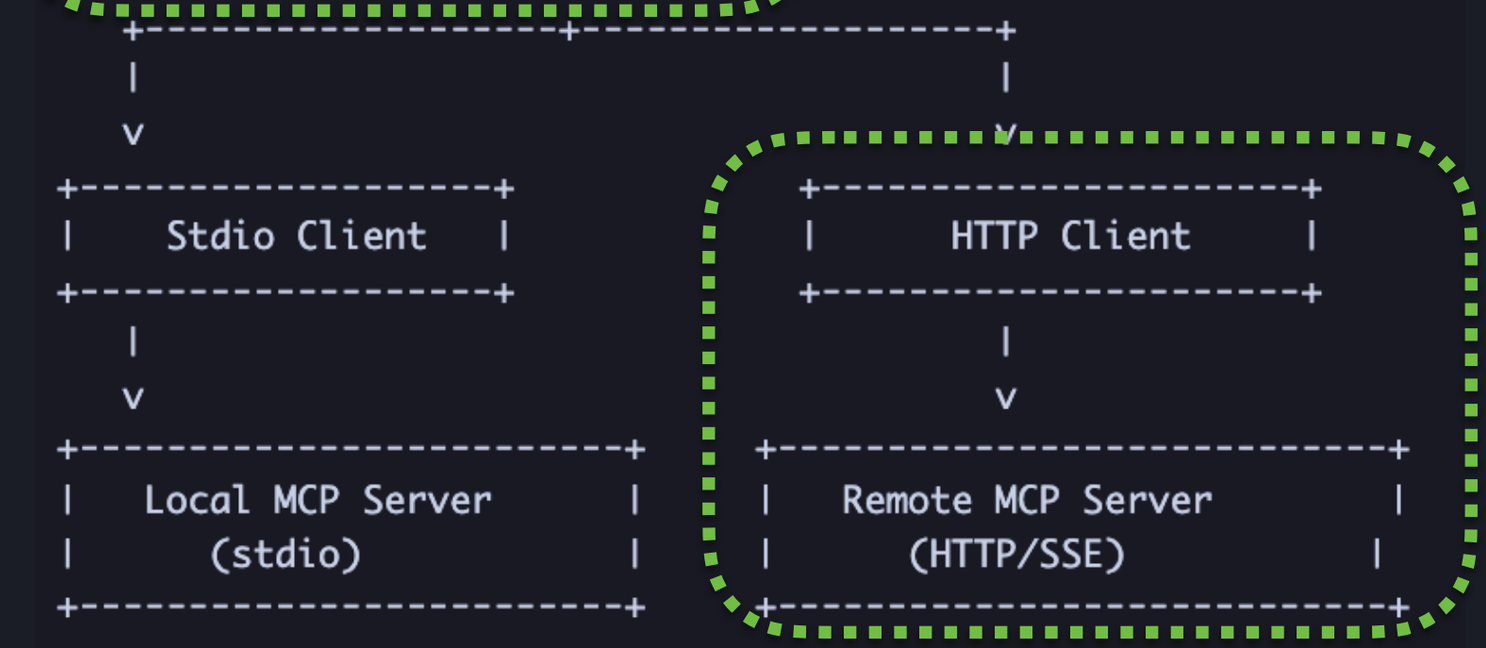
Technical Architecture


kubectl-ai
Agent (Conversation)
• Integrate MCP Manager
• Tool discovery & register
• AI inference engine
MCP Manager
(manager.go)
• Manage MCP Client connections
• Tool discovery & aggregation
• Connection lifecycle
• Status monitoring
Stdio Client
HTTP Client
Local MCP Server
(stdio)
Remote MCP Server
(HTTP/SSE)



Demo 01

Demo 02



Demo 03

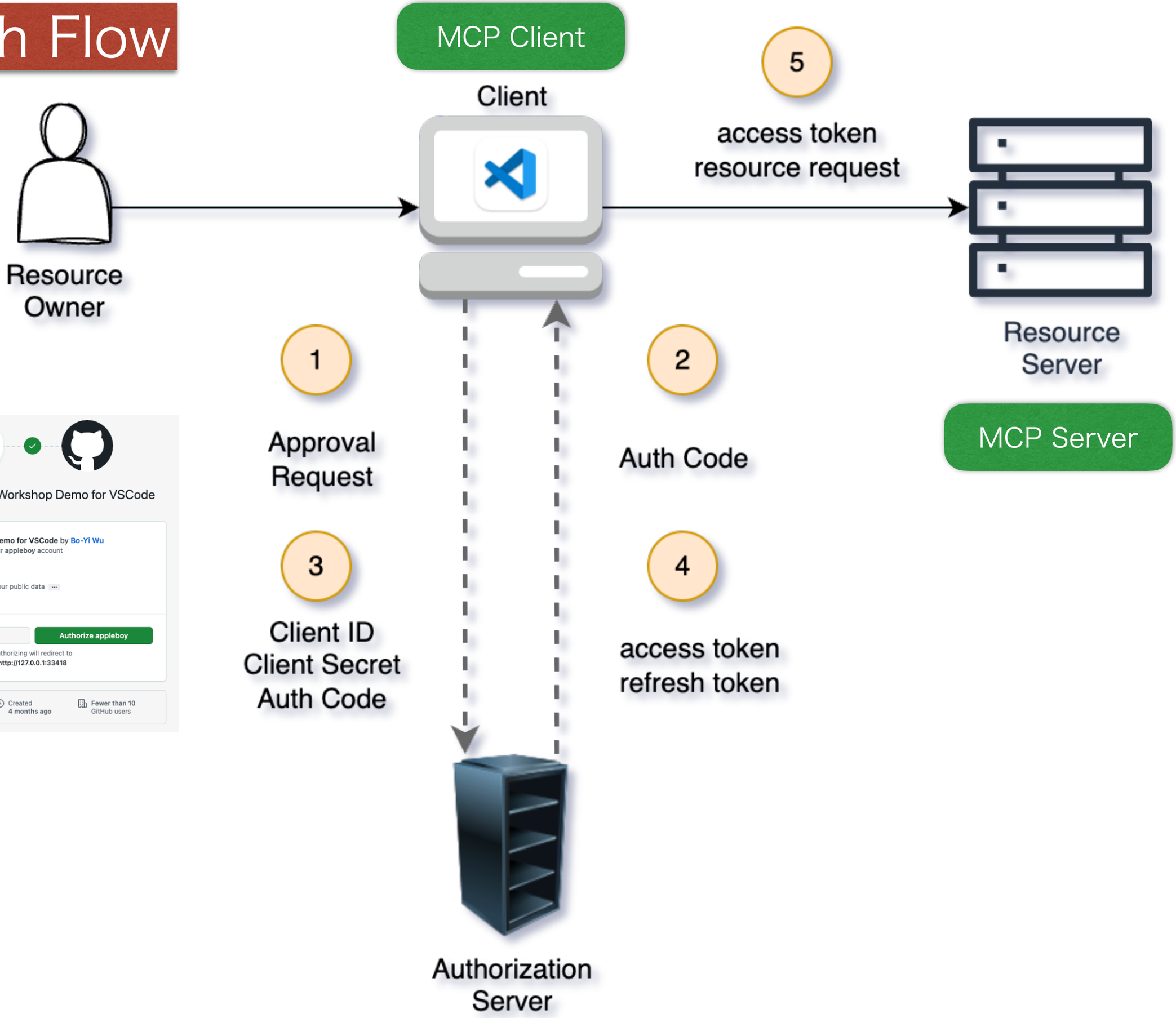
MCP Manager

HTTPClient and StdioClient

Three Authentication Mode

- Bearer Token
- Basic Auth
- API Key (custom header)

OAuth Flow

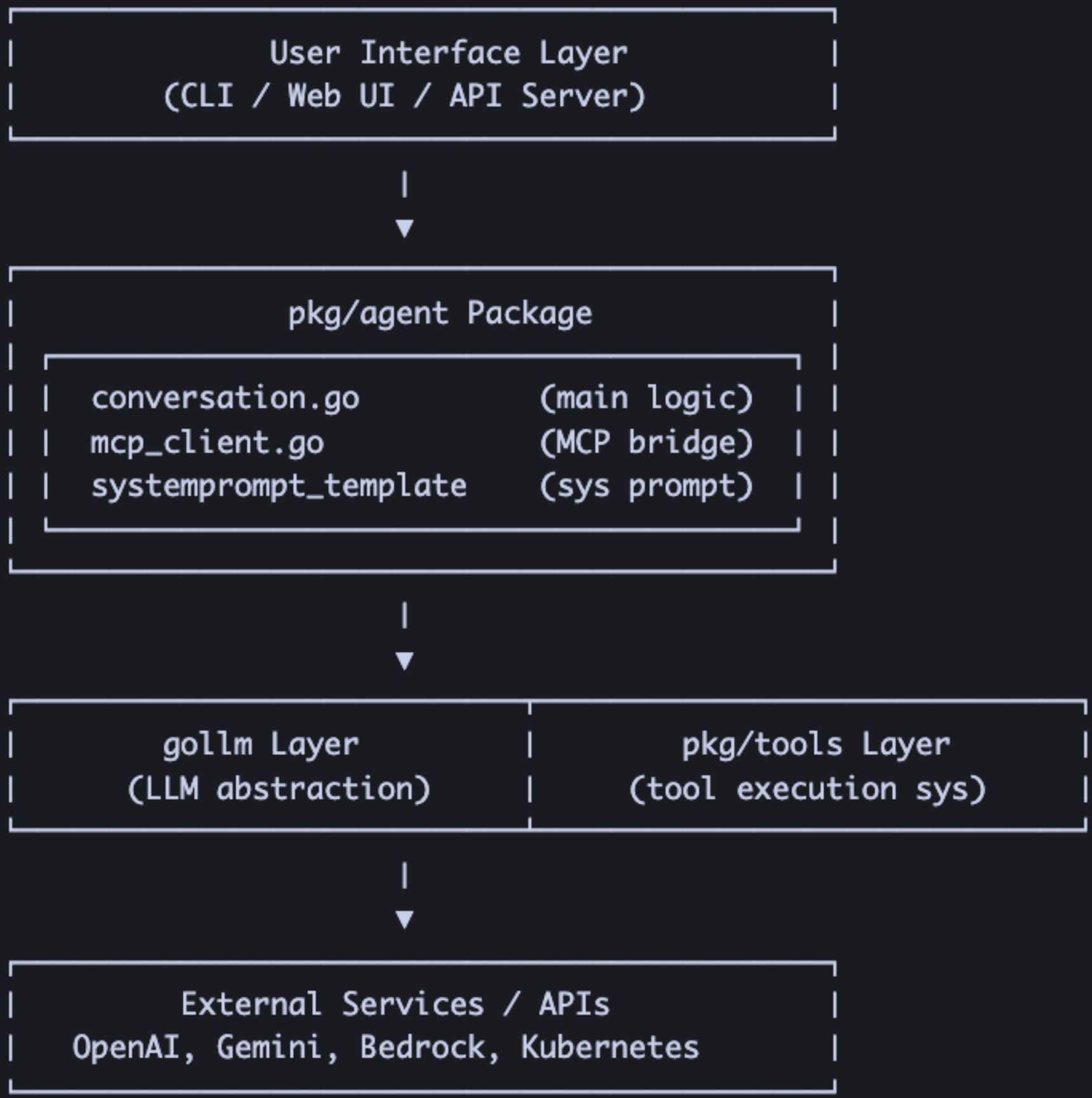

MCP Client
Client
5
access token
resource request
Resource
Owner
Resource
Server
1
Approval
Request
2
Auth Code
MCP Server
3
Client ID
Client Secret
Auth Code
4
access token
refresh token
Authorization
Server

1

Authorize MCP Workshop Demo for VSCode
MCP Workshop Demo for VSCode by Bo-Yi Wu
wants to access your appleboy account
Public data only
Limited access to your public data
Cancel
Authorize appleboy
Authorizing will redirect to
http://127.0.0.1:33418
Not owned or operated by GitHub
Created 4 months ago
Fewer than 10 GitHub users

Initial MCP Manger

1. Start `kubectl-ai --mcp-client`
   ↓
2. `InitializeManager()`
   ├── `LoadConfig("~/.config/kubectl-ai/mcp.yaml")`
   │ ├── File does not exist? → Create default configuration
   │ └── Apply environment variable overrides
   └── `NewManager(config)`
   ↓
3. `DiscoverAndConnectServers(ctx)`
   ├── `ConnectAll(ctx)` [connect to all servers in parallel]
   │ ├── `NewClient(serverConfig)`
   │ │ ├── If URL exists → `NewHTTPClient`
   │ │ └── Otherwise → `NewStdioClient`
   │ └── `client.Connect(ctx)`
   │ ├── Establish connection
   │ ├── Initialize handshake
   │ └── Validate connection
   └── Wait to stabilize (2 seconds)
   ↓
4. `RefreshToolDiscovery(ctx)` [with retries]
   ├── `ListAvailableTools(ctx)`
   │ └── For each client: `client.ListTools(ctx)`
   └── Record discovered tools
   ↓
5. `RegisterTools(ctx, callback)`
   └── For each tool, call `callback` to register it with the Agent


User Interface Layer
(CLI / Web UI / API Server)
pkg/agent Package
conversation.go (main logic)
mcp_client.go (MCP bridge)
systemprompt_template (sys prompt)
gollm Layer
(LLM abstraction)
pkg/tools Layer
(tool execution sys)
External Services / APIs
OpenAI, Gemini, Bedrock, Kubernetes

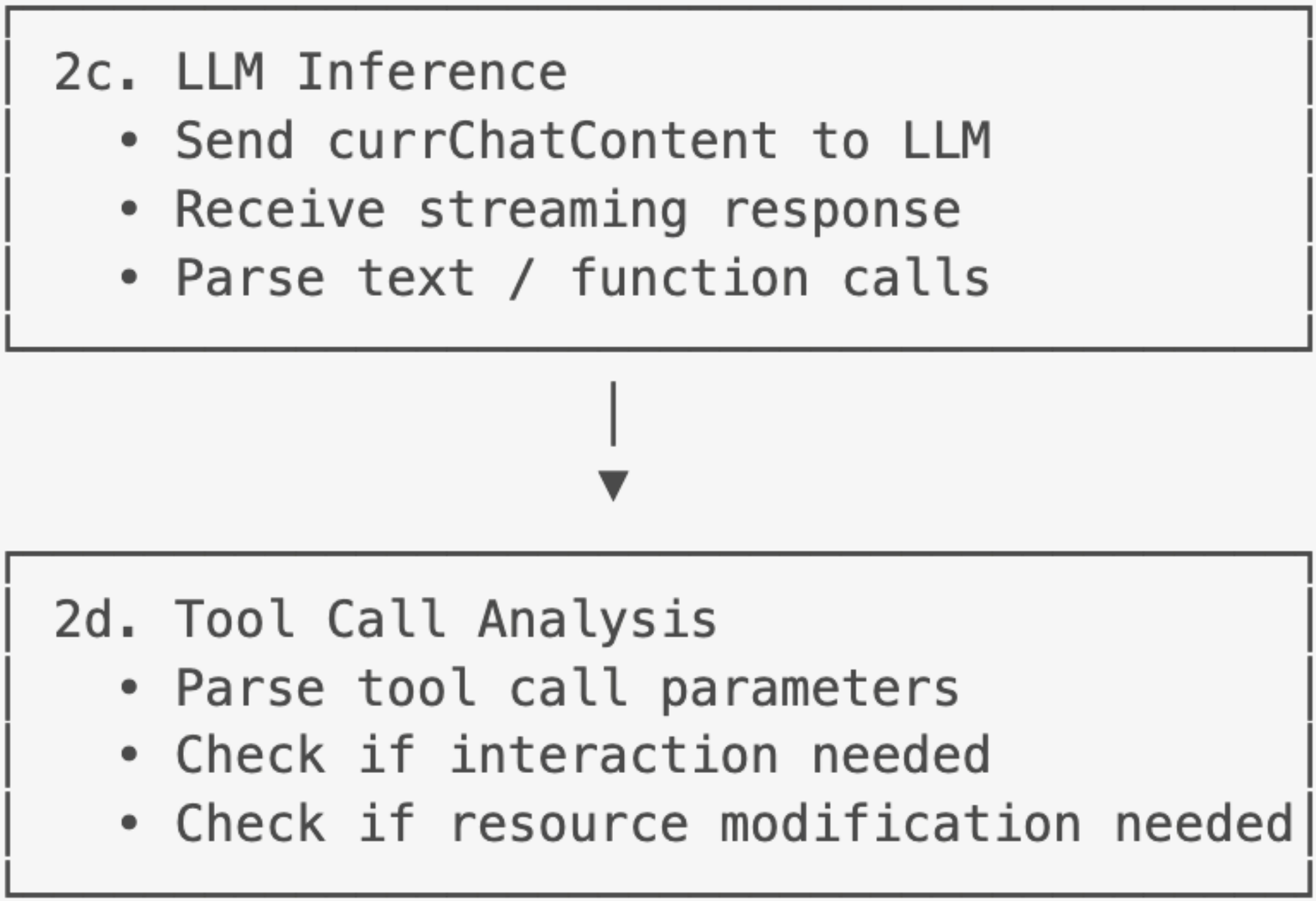
Agent Structure

```go
type Agent struct {
    // === Communication Channels ===
    Input  chan any  // Receives UI input
    Output chan any  // Sends messages to UI
    // === Execution Mode ===
    RunOnce      bool   // Non-interactive mode flag
    InitialQuery string // Initial query
    // === Agentic Loop State ===
    pendingFunctionCalls []ToolCallAnalysis  // Pending tool executions
    currChatContent      []any               // Current chat contents
    currIteration        int                 // Current iteration count
    // === LLM Integration ===
    LLM         gollm.Client // LLM client
    llmChat     gollm.Chat   // Chat session
    Model       string       // Model name
    Provider    string       // Provider (openai/gemini/bedrock)
    // === Tool System ===
    Tools             tools.Tools  // Tool registry
    EnableToolUseShim bool         // Tool use shim mode
    // === MCP Support ===
    MCPClientEnabled bool         // MCP client enabled
    mcpManager       *mcp.Manager // MCP manager
    // === Session Management ===
    session          *api.Session         // Current session
    ChatMessageStore api.ChatMessageStore // Message persistence
    sessionMu        sync.Mutex           // Session lock
    // === Permissions & Configurations ===
    SkipPermissions bool   // Skip permission checks
    MaxIterations   int    // Maximum iterations
    Kubeconfig      string // Kubernetes config path
    workDir         string // Working directory
}
```

```
1. Initialization Phase
   • Create temporary working directory
   • Generate System Prompt
   • Initialize LLM Chat Session
   • Load Tool Definitions (Function Definitions)
   • Connect to MCP Server (if enabled)
```

2a. State Check
- Idle/Done → Wait for user input
- WaitingForInput → Await confirmation
- Running → Execute agent logic

2b. Meta Query Handling
- clear/reset → Clear conversation
- model/models → Show model info
- tools → List available tools
- session → Session management


2c. LLM Inference
• Send currChatContent to LLM
• Receive streaming response
• Parse text / function calls
2d. Tool Call Analysis
• Parse tool call parameters
• Check if interaction needed
• Check if resource modification needed

2e. Permission Check
- ModifiesResource == "yes"?
- SkipPermissions == false?
- → Request user confirmation

▼

2f. Tool Execution
- DispatchToolCalls()
- Collect execution results
- Update currChatContent

▼

2g. Iteration Control
- currIteration++
- Check MaxIterations
- No tool calls → Set to Done

```go
type ToolCallAnalysis struct {
  FunctionCall        gollm.FunctionCall  // Original function call returned by LLM
  ParsedToolCall      *tools.ToolCall     // Parsed tool invocation
  IsInteractive       bool                // Whether interactive input is required
  IsInteractiveError  error               // Error during interactive check
  ModifiesResourceStr string             // "yes"/"no"/"unknown"
}

func (c *Agent) analyzeToolCalls(ctx context.Context,
  toolCalls []gollm.FunctionCall) ([]ToolCallAnalysis, error) {

  toolCallAnalysis := make([]ToolCallAnalysis, len(toolCalls))
  for i, call := range toolCalls {
    toolCallAnalysis[i].FunctionCall = call

    // Parse tool invocation
    toolCall, err := c.Tools.ParseToolInvocation(ctx,
      call.Name, call.Arguments)
    if err != nil {
      return nil, fmt.Errorf("parsing tool call: %w", err)
    }

    // Check whether interaction is needed (e.g., vim, nano, etc.)
    toolCallAnalysis[i].IsInteractive, err =
      toolCall.GetTool().IsInteractive(call.Arguments)
    if err != nil {
      toolCallAnalysis[i].IsInteractiveError = err
    }

    // Check whether resources are modified (requires permission confirmation)
    toolCallAnalysis[i].ModifiesResourceStr =
      toolCall.GetTool().CheckModifiesResource(call.Arguments)

    toolCallAnalysis[i].ParsedToolCall = toolCall
  }
  return toolCallAnalysis, nil
}
```

✅ Safe to execute

⚠️ User confirmation required

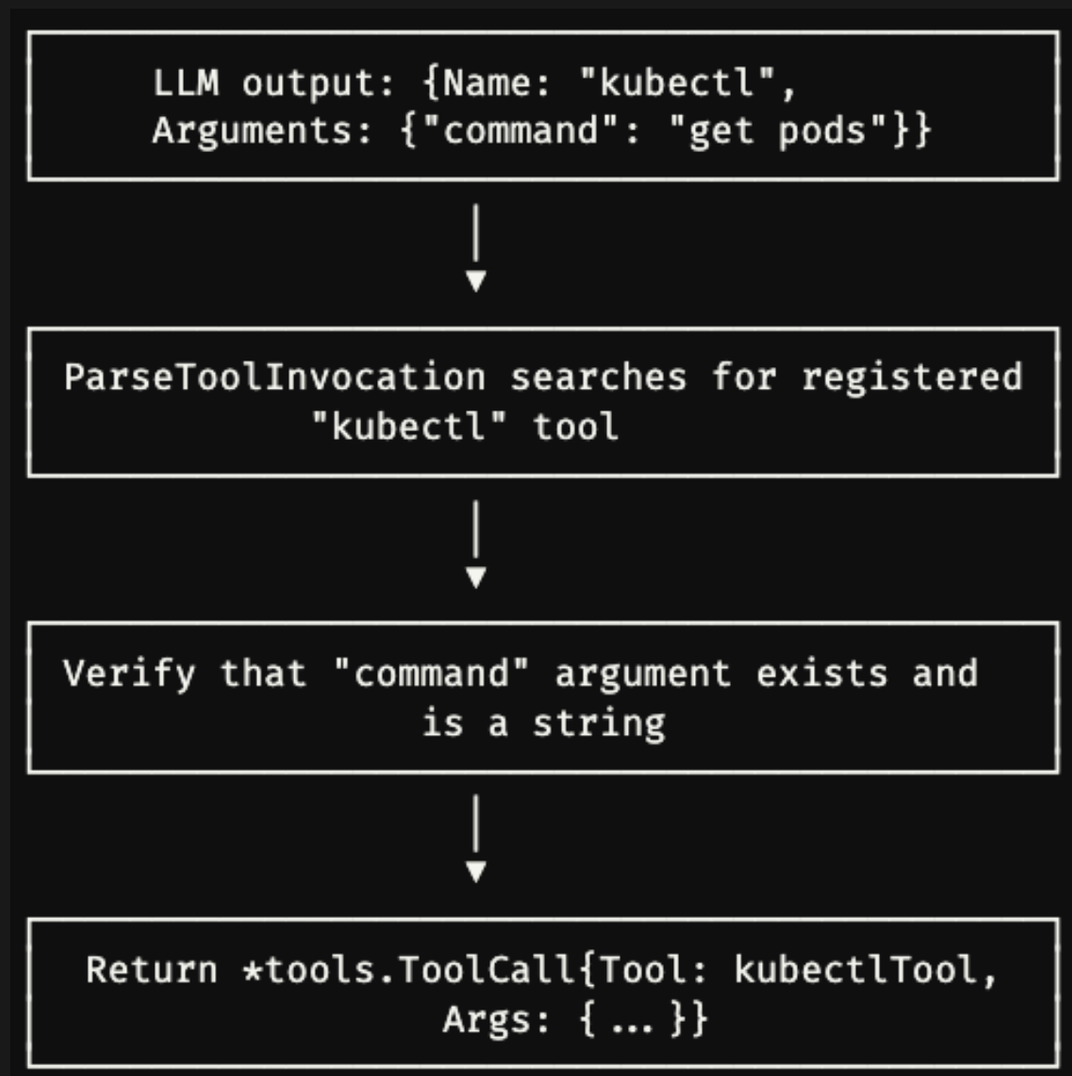
🚫 Should be blocked

1

```
gollm.FunctionCall{
    ID:        "call_abc123",
    Name:      "kubectl",
    Arguments: map[string]any{
        "command": "delete pod nginx-abc -n production",
    },
}
```

2

```
toolCall, err := c.Tools.ParseToolInvocation(ctx, call.Name, call.Arguments)
if err != nil {
    return nil, fmt.Errorf("error parsing tool call: %w", err)
}
```



3

```
toolCallAnalysis[i].IsInteractive, err = toolCall.GetTool().IsInteractive(call.Arguments)
if err != nil {
    toolCallAnalysis[i].IsInteractiveError = err
}
```

Why need to check interactive command?

- 🚫 Stuck: Agent keeps waiting for user input
- 🚫 Can’t get output: Terminal took over
- 🚫 Workflow broken: Agent loop got interrupted

4

```
toolCallAnalysis[i].ModifiesResourceStr =
  toolCall.GetTool().CheckModifiesResource(call.Arguments)
```

Why need to check modify resource command?

```
writeOps = map[string]bool{
  "create": true, "apply": true, "edit": true, "delete": true,
  "patch": true, "replace": true, "scale": true, "autoscale": true,
  "expose": true, "run": true, "exec": true, "set": true,
  "label": true, "annotate": true, "taint": true, "drain": true,
  "cordon": true, "uncordon": true, "debug": true, "attach": true,
  "cp": true, "reconcile": true, "approve": true, "deny": true,
  "certificate": true,
}
```

1. Interactive Check (IsInteractive)	→ Prevents hanging
2. Resource Modification Check (ModifiesResource)	→ Requires confirmation
3. Permission Flag (SkipPermissions)	→ Bypass in CI/CD

Conversation Flow

```
LLM: "I want to use kubectl edit to modify the deployment."
      |
      ▼
Agent: "kubectl edit is an interactive command and is not allowed."
      |
      ▼
LLM: "Okay, then I'll use kubectl patch deployment nginx --patch=' ... '"
      |
      ▼
Agent: "kubectl patch requires confirmation. Do you want to continue?"
      |
      ▼
User: "Yes"
      |
      ▼
Agent: Executed successfully ✅
```

Thanks